

Pythonia

Manual de ruta de especialización

Bases de datos

Ruta de especialización

Datos ordenados, consultas rápidas y aplicaciones confiables.



6 módulos | 12 lecciones | 12 semanas en paralelo al tronco

Repasa, practica y registra cada ejecución en tu Cuadernillo-Diario.

Antes de empezar

Diseñar y consultar bases de datos relacionales con SQL, sqlite3, SQLAlchemy y PostgreSQL, y analizar grandes volúmenes con DuckDB.

Esta ruta crece junto al tronco común. Cada módulo se desbloquea cuando completas sus lecciones requisito del tronco, así nunca llegas a la especialidad con la base floja. En la app, el módulo aparece en el nivel indicado del árbol; si ya cumples los requisitos antes, puedes adelantarlo.

Todo el código de este manual fue ejecutado: la salida que ves es la real. Cada lección incluye una **práctica evaluable** centrada en la lógica de la especialidad, sin librerías externas, para que puedas corregirla en la app. Cada módulo cierra con una **misión** y la ruta termina con un **proyecto final** (el jefe de la temporada).

Tu plan

Módulo	Aparece en	Requisitos del tronco
1. SQL esencial SELECT, WHERE y ORDER BY JOIN y GROUP BY	Nivel 2	N2.4 Archivos, CSV y JSON
2. sqlite3 desde Python Parámetros seguros y filas como diccionarios Transacciones y errores	Nivel 2	N2.3 Módulos y paquetes N2.5 Excepciones a fondo
3. Modelado de datos Tablas, claves y relaciones Índices y planes de consulta	Nivel 3	N2.5 Excepciones a fondo
4. SQLAlchemy ORM Modelos y sesiones Relaciones y consultas agregadas	Nivel 3	N2.7 Programación orientada a objetos II N3.3 Context managers N3.4 Type hints y dataclasses
5. PostgreSQL y arquitectura de datos PostgreSQL desde Python Patrón Repository	Nivel 4	N3.4 Type hints y dataclasses N4.7 Patrones de diseño pythónicos
6. Análisis con DuckDB Consultar archivos directamente DuckDB, pandas y Parquet	Nivel 4	N4.5 Profiling y optimización N2.4 Archivos, CSV y JSON
Proyecto final: Sistema de inventario con reportes	Al cerrar	Todos los módulos

Ritmo sugerido: 30 a 45 minutos al día, 5 días por semana. Lo importante es no cortar la racha.

Módulo 1

SQL esencial

Requisitos: N2.4 Archivos, CSV y JSON. Aparece en el Nivel 2 del árbol.

1 SELECT, WHERE y ORDER BY

SQL es el lenguaje de las bases de datos relacionales. Una tabla tiene filas (registros) y columnas (campos).

SELECT columnas FROM tabla WHERE condición ORDER BY columna es la consulta más usada del mundo. COUNT, SUM y AVG resumen datos.

SQLite viene incluido en Python: con `sqlite3.connect(":memory:")` tienes una base para practicar al instante.

Ejemplo

```

1  import sqlite3
2
3  con = sqlite3.connect(":memory:")
4  con.executescript("""
5  CREATE TABLE productos (id INTEGER PRIMARY KEY, nombre TEXT, categoria TEXT, precio
   ↳ REAL, stock INTEGER);
6  INSERT INTO productos (nombre, categoria, precio, stock) VALUES
7  ('Cable 2mm', 'Electricidad', 3.2, 150), ('Pintura blanca', 'Pinturas', 12.5, 40),
8  ('Tornillo', 'Ferretería', 0.15, 2000), ('Bombillo LED', 'Electricidad', 2.8, 0),
9  ('Rodillo', 'Pinturas', 6.0, 25);
10 """)
11
12 consulta = """
13 SELECT nombre, precio FROM productos
14 WHERE categoria = 'Electricidad' OR precio > 10
15 ORDER BY precio DESC
16 """
17 for fila in con.execute(consulta):
18     print(fila)
19 print("Sin stock:", con.execute("SELECT COUNT(*) FROM productos WHERE stock =
   ↳ 0").fetchone()[0])

```

Salida real

```

('Pintura blanca', 12.5)
('Cable 2mm', 3.2)
('Bombillo LED', 2.8)
Sin stock: 1

```

Consejos

- En SQL los textos van entre comillas simples: 'Pinturas'.
- Practica con la extensión SQLite Viewer de VS Code o con DB Browser for SQLite.

Ejercicio

Consulta los productos con stock entre 1 y 50 ordenados por nombre, y el precio promedio por categoría.

Práctica evaluable**Un SELECT escrito a mano** · La lógica de la lección, sin librerías externas

Para entender qué hace el motor con tu SQL, impleméntalo en Python. Ya existe PRODUCTOS, una lista de diccionarios (una tabla). Escribe seleccionar(tabla, columnas=None, donde=None, orden=None, desc=False):

- donde: una función que recibe una fila y devuelve True si se queda (WHERE);
- orden: el nombre de la columna para ordenar (ORDER BY), descendente si desc;
- columnas: lista de columnas a devolver (SELECT col1, col2); si es None, todas.

Devuelve una lista de diccionarios nuevos y no modifica la tabla.

Datos que prepara el evaluador (también prueba con otros valores)

```

1  PRODUCTOS = [
2      {"id": 1, "nombre": "Cable 2mm", "categoria": "Electricidad", "precio": 3.2,
3      ↪ "stock": 150},
4      {"id": 2, "nombre": "Pintura blanca", "categoria": "Pinturas", "precio": 12.5,
5      ↪ "stock": 40},
6      {"id": 3, "nombre": "Tornillo", "categoria": "Ferretería", "precio": 0.15,
7      ↪ "stock": 2000},
8      {"id": 4, "nombre": "Bombillo LED", "categoria": "Electricidad", "precio": 2.8,
9      ↪ "stock": 0},
10     {"id": 5, "nombre": "Rodillo", "categoria": "Pinturas", "precio": 6.0, "stock":
11     ↪ 25},
12 ]

```

Plantilla para empezar

```

1  def seleccionar(tabla, columnas=None, donde=None, orden=None, desc=False):
2      """SELECT columnas FROM tabla WHERE donde ORDER BY orden"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué cláusula filtra filas en SQL?

- SELECT
- WHERE
- ORDER BY
- FROM

2 JOIN y GROUP BY

Los datos se reparten en tablas relacionadas. JOIN las une por una columna común; LEFT JOIN conserva las filas de la izquierda aunque no tengan pareja.

GROUP BY agrupa filas para resumirlas: total de ventas por sucursal, cantidad de pedidos por cliente.

HAVING filtra después de agrupar (por ejemplo, sucursales con más de 100 en ventas); WHERE filtra antes.

Ejemplo

```

1  import sqlite3
2
3  con = sqlite3.connect(":memory:")
4  con.executescript("""
5  CREATE TABLE sucursales (id INTEGER PRIMARY KEY, nombre TEXT);
6  CREATE TABLE ventas (id INTEGER PRIMARY KEY, sucursal_id INTEGER, monto REAL);
7  INSERT INTO sucursales VALUES (1, 'Centro'), (2, 'Norte'), (3, 'Sur');
8  INSERT INTO ventas (sucursal_id, monto) VALUES (1, 120), (1, 80), (2, 200), (1,
9  50), (2, 30);
10 """)
11
12 consulta = """
13 SELECT s.nombre, COUNT(v.id) AS ventas, COALESCE(SUM(v.monto), 0) AS total
14 FROM sucursales s
15 LEFT JOIN ventas v ON v.sucursal_id = s.id
16 GROUP BY s.id
17 ORDER BY total DESC
18 """
19
20 for fila in con.execute(consulta):
21     print(fila)

```

Salida real

```

('Centro', 3, 250.0)
('Norte', 2, 230.0)
('Sur', 0, 0)

```

Consejos

- Usa alias cortos (s, v) para que las consultas con JOIN sean legibles.
- Con LEFT JOIN, la sucursal Sur aparece aunque no tenga ventas: por eso usamos COALESCE.

Ejercicio

Agrega una tabla productos y obtén el producto más vendido de cada sucursal.

Práctica evaluable

LEFT JOIN y GROUP BY a mano · La lógica de la lección, sin librerías externas

Escribe `totales_por_sucursal(sucursales, ventas)`: `sucursales` es una lista de (`id`, `nombre`) y `ventas` una lista de (`sucursal_id`, `monto`). Devuelve una lista de tuplas (`nombre`, `cantidad_de_ventas`, `total`) que incluya las sucursales sin ventas (con 0 y 0, como un LEFT JOIN), ordenada por total de mayor a menor y, si empatan, por nombre.

Plantilla para empezar

```

1 def totales_por_sucursal(sucursales, ventas):
2     """Equivale a un LEFT JOIN con GROUP BY."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Si una sucursal no tiene ventas, ¿qué JOIN la muestra igual?

- a) INNER JOIN
- b) Ninguno
- c) CROSS JOIN
- d) LEFT JOIN desde sucursales

Misión del módulo

Responde 10 preguntas de negocio con SQL sobre una base de ventas de ejemplo.

Módulo 2

sqlite3 desde Python

Requisitos: N2.3 Módulos y paquetes, N2.5 Excepciones a fondo. Aparece en el Nivel 2 del árbol.

3 Parámetros seguros y filas como diccionarios

Nunca armes SQL concatenando textos del usuario: es la puerta de la inyección SQL. Usa parámetros ? y pasa los valores aparte.

`executemany` inserta muchas filas de una vez; `commit()` confirma los cambios en el archivo.

Con `row_factory = sqlite3.Row` cada fila se accede por nombre de columna, como un diccionario.

Ejemplo

```

1 import sqlite3
2
3 con = sqlite3.connect("tienda.db")
4 con.row_factory = sqlite3.Row
5 con.execute("CREATE TABLE IF NOT EXISTS clientes (id INTEGER PRIMARY KEY, nombre
6 ↵ TEXT, ciudad TEXT)")
7 nuevos = [("Ana", "Lima"), ("Luis", "Quito"), ("Marta", "Lima")]
8 con.executemany("INSERT INTO clientes (nombre, ciudad) VALUES (?, ?)", nuevos)
9 con.commit()
10
11 ciudad = "Lima" # imagina que viene del usuario
12 for f in con.execute("SELECT * FROM clientes WHERE ciudad = ?", (ciudad,)):
13     print(f["id"], f["nombre"], dict(f))
14
15 malicioso = "x' OR '1'='1"
16 total = con.execute("SELECT COUNT(*) FROM clientes WHERE ciudad = ?",
17 ↵ (malicioso,)).fetchone()[0]
18 print("Filas devueltas al intento de inyección:", total)
19 con.close()

```

Salida real

```
1 Ana {'id': 1, 'nombre': 'Ana', 'ciudad': 'Lima'}
3 Marta {'id': 3, 'nombre': 'Marta', 'ciudad': 'Lima'}
Filas devueltas al intento de inyección: 0
```

Consejos

- Fíjate en (ciudad,): un solo parámetro igual va dentro de una tupla.
- Los nombres de tablas y columnas no pueden ser parámetros; si son dinámicos, válídalos contra una lista permitida.

Ejercicio

Agrega funciones buscar_por_nombre, actualizar_ciudad y eliminar_cliente, todas con parámetros.

Práctica evaluable

Consultas parametrizadas seguras · La lógica de la lección, sin librerías externas

Escribe armar_consulta(tabla, filtros, permitidas), que devuelve la tupla (sql, parametros) lista para con.execute:

```
armar_consulta("clientes", {"ciudad": "Lima", "activo": 1}, {"clientes"}) →
("SELECT * FROM clientes WHERE ciudad = ? AND activo = ?", ("Lima", 1))
```

Sin filtros, no hay WHERE. Los nombres de tablas y columnas no pueden ser parámetros, así que válídalos: la tabla debe estar en permitidas y cada columna debe ser un identificador válido (str.isidentifier()); si no, ValueError. Los valores nunca se pegan en el texto.

Plantilla para empezar

```
1 def armar_consulta(tabla, filtros, permitidas):
2     """-> (sql, parametros)"""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cómo se previene la inyección SQL?

- Escribiendo SQL en mayúsculas
- Con f-strings
- Con parámetros ? y valores separados
- Con comentarios

4 Transacciones y errores

Una transacción agrupa operaciones que deben ocurrir todas o ninguna, como una transferencia: restar en una cuenta y sumar en otra.

Usar la conexión como context manager (`with con:`) confirma si todo sale bien y revierte (`rollback`) si hay una excepción.

Las restricciones `CHECK`, `UNIQUE` y `NOT NULL` hacen que la base misma rechace datos inválidos con `IntegrityError`.

Ejemplo

```

1  import sqlite3
2
3  con = sqlite3.connect(":memory:")
4  con.execute("CREATE TABLE cuentas (titular TEXT PRIMARY KEY, saldo REAL CHECK
↳ (saldo >= 0))")
5  con.executemany("INSERT INTO cuentas VALUES (?, ?)", [("Ana", 100), ("Luis", 50)])
6  con.commit()
7
8  def transferir(origen, destino, monto):
9      try:
10         with con: # commit si todo sale bien, rollback si hay error
11             con.execute("UPDATE cuentas SET saldo = saldo + ? WHERE titular = ?",
↳ (monto, destino))
12             con.execute("UPDATE cuentas SET saldo = saldo - ? WHERE titular = ?",
↳ (monto, origen))
13             print(f"OK: {origen} -> {destino} {monto}")
14         except sqlite3.IntegrityError as e:
15             print(f"Revertido: {e}")
16
17  transferir("Ana", "Luis", 30)
18  transferir("Luis", "Ana", 500)
19  print(con.execute("SELECT * FROM cuentas").fetchall())

```

Salida real

```

OK: Ana -> Luis 30
Revertido: CHECK constraint failed: saldo >= 0
[('Ana', 70.0), ('Luis', 80.0)]

```

Consejos

- Fíjate que en la transferencia fallida tampoco se sumó a Ana: el `rollback` deshizo ambas operaciones.
- Deja que la base valide lo esencial; tu código valida además para dar mensajes amables.

Ejercicio

Registra cada transferencia en una tabla movimientos dentro de la misma transacción.

Práctica evaluable

Una transferencia atómica · La lógica de la lección, sin librerías externas

Escribe `transferir(cuentas, origen, destino, monto)` sobre un diccionario `{titular: saldo}`. Todo o nada: si el monto no es positivo, si alguna cuenta no existe o si el saldo no alcanza, lanza `ValueError` y deja el diccionario intacto. Si todo está bien, modifica ambas cuentas y devuelve el diccionario.

Plantilla para empezar

```
1 def transferir(cuentas, origen, destino, monto):
2     """Todo o nada, como una transacción."""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Si falla la segunda operación dentro de `with con:`, ¿qué pasa con la primera?

- a) Queda guardada
- b) Se duplica
- c) Se revierte también
- d) Depende del sistema

Misión del módulo

Crea una agenda de clientes de consola con búsqueda, alta, edición y baja.

Módulo 3

Modelado de datos

Requisitos: N2.5 Excepciones a fondo. Aparece en el Nivel 3 del árbol.

5

Tablas, claves y relaciones

Modelar es decidir qué tablas existen y cómo se relacionan. Cada tabla tiene una clave primaria; las claves foráneas apuntan a otras tablas.

Las relaciones uno a muchos (una categoría, muchos productos) usan una clave foránea; las de muchos a muchos (pedidos y productos) usan una tabla puente.

Normalizar es no repetir datos: el nombre de una categoría se guarda una sola vez y se referencia por su id.

Ejemplo

```

1  import sqlite3
2
3  con = sqlite3.connect(":memory:")
4  con.execute("PRAGMA foreign_keys = ON")
5  con.executescript("""
6  CREATE TABLE categorias (id INTEGER PRIMARY KEY, nombre TEXT NOT NULL UNIQUE);
7  CREATE TABLE productos (
8      id INTEGER PRIMARY KEY, nombre TEXT NOT NULL,
9      precio REAL NOT NULL CHECK (precio > 0),
10     categoria_id INTEGER NOT NULL REFERENCES categorias(id));
11  CREATE TABLE pedidos (id INTEGER PRIMARY KEY, fecha TEXT NOT NULL);
12  CREATE TABLE pedido_items (          -- tabla puente: muchos a muchos
13     pedido_id INTEGER REFERENCES pedidos(id),
14     producto_id INTEGER REFERENCES productos(id),
15     cantidad INTEGER NOT NULL CHECK (cantidad > 0),
16     PRIMARY KEY (pedido_id, producto_id));
17  INSERT INTO categorias (nombre) VALUES ('Pinturas');
18  INSERT INTO productos (nombre, precio, categoria_id) VALUES ('Rodillo', 6.0, 1);
19  """)
20
21  try:
22     con.execute("INSERT INTO productos (nombre, precio, categoria_id) VALUES
23         ↪ ('Fantasma', 1, 99)")
24  except sqlite3.IntegrityError as e:
25     print("Bloqueado:", e)
26  print([r[0] for r in con.execute("SELECT name FROM sqlite_master WHERE type =
27     ↪ 'table'")])

```

Salida real

```

Bloqueado: FOREIGN KEY constraint failed
['categorias', 'productos', 'pedidos', 'pedido_items']

```

Consejos

- En SQLite debes activar `PRAGMA foreign_keys = ON` en cada conexión; si no, no se validan.
- Dibuja el modelo antes de crearlo: dbdiagram.io o un papel bastan.

Ejercicio

Agrega clientes al modelo (un cliente tiene muchos pedidos) e inserta un pedido completo con tres productos.

Práctica evaluable

Integridad referencial · La lógica de la lección, sin librerías externas

Las tablas se relacionan por claves. Escribe:

- `claves_rotas(productos, categorias)`: nombres de los productos cuyo `categoria_id` no existe entre los `id` de `categorias` (lo que una `FOREIGN KEY` impediría).
- `detalle_pedido(pedido_id, items, productos)`: con la tabla puente `items` (tuplas (`pedido_id`, `producto_id`, `cantidad`)), devuelve `[(nombre, cantidad), ...]` del pedido indicado.

Plantilla para empezar

```

1 def claves_rotas(productos, categorias):
2     """Productos con categoría inexistente."""
3
4
5 def detalle_pedido(pedido_id, items, productos):
6     """[(nombre, cantidad)] de un pedido."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cómo se modela una relación muchos a muchos?

- a) Duplicando tablas
- b) No se puede
- c) Con una columna de texto separada por comas
- d) Con una tabla puente con dos claves foráneas

6 Índices y planes de consulta

Sin índice, la base revisa todas las filas para encontrar las que buscas (escaneo completo). Un índice es como el índice de un libro: salta directo.

EXPLAIN QUERY PLAN muestra cómo piensa ejecutar la consulta. Es la herramienta principal para diagnosticar lentitud.

Los índices aceleran lecturas pero ocupan espacio y hacen más lentas las escrituras: crea los que tus consultas realmente usan.

Ejemplo

```

1 import sqlite3, random
2
3 con = sqlite3.connect(":memory:")
4 con.execute("CREATE TABLE ventas (id INTEGER PRIMARY KEY, cliente TEXT, monto
5     ↪ REAL)")
6 random.seed(1)
7 con.executemany("INSERT INTO ventas (cliente, monto) VALUES (?, ?)",
8     [(f"cliente{random.randint(1, 5000)}", random.random() * 100) for _
9     ↪ in range(50_000)])
10
11 consulta = "SELECT SUM(monto) FROM ventas WHERE cliente = 'cliente42'"
12 plan = lambda: con.execute("EXPLAIN QUERY PLAN " + consulta).fetchall()[0][-1]
13 print("Sin índice:", plan())
14 con.execute("CREATE INDEX idx_ventas_cliente ON ventas(cliente)")
15 print("Con índice:", plan())

```

Salida real

```

Sin índice: SCAN ventas
Con índice: SEARCH ventas USING INDEX idx_ventas_cliente (cliente=?)

```

Consejos

- Indexa columnas usadas en WHERE, JOIN y ORDER BY frecuentes.
- SCAN significa recorrer toda la tabla; SEARCH ... USING INDEX es lo que buscas.

Ejercicio

Mide con timeit la consulta antes y después del índice y anota la diferencia en tu cuadernillo.

Práctica evaluable

Un índice a mano · La lógica de la lección, sin librerías externas

Un índice evita recorrer toda la tabla. Escribe:

- crear_indice(filas, columna): diccionario {valor: [posiciones de las filas con ese valor]}.
- buscar(filas, indice, valor): las filas con ese valor, usando el índice (sin recorrer la tabla). Si el valor no existe, [].

Plantilla para empezar

```

1 def crear_indice(filas, columna):
2     """{valor: [posiciones]}"""
3
4
5 def buscar(filas, indice, valor):
6     """Filas con ese valor, usando el índice."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

En EXPLAIN QUERY PLAN, ¿qué indica SCAN?

- Que hay un error
- Que recorre toda la tabla
- Que la consulta está en caché
- Que usa un índice

Misión del módulo

Diseña el modelo completo de una tienda (clientes, productos, pedidos) y documenta su diagrama.

Módulo 4

SQLAlchemy ORM

Requisitos: N2.7 Programación orientada a objetos II, N3.3 Context managers, N3.4 Type hints y dataclasses. Aparece en el Nivel 3 del árbol.

7 Modelos y sesiones

Un ORM (mapeo objeto-relacional) te deja trabajar con clases de Python en lugar de escribir SQL a mano. SQLAlchemy es el estándar en Python.

En SQLAlchemy 2.0 cada modelo hereda de `DeclarativeBase` y declara columnas con `Mapped[tipo]` y `mapped_column`: es tipado y amigable con mypy.

La `Session` agrupa cambios y los confirma con `commit()`; `select()` construye consultas que se ejecutan con `scalars()`.

Ejemplo

```

1  from sqlalchemy import create_engine, select, String
2  from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session
3
4  class Base(DeclarativeBase):
5      pass
6
7  class Producto(Base):
8      __tablename__ = "productos"
9      id: Mapped[int] = mapped_column(primary_key=True)
10     nombre: Mapped[str] = mapped_column(String(80))
11     precio: Mapped[float]
12     stock: Mapped[int] = mapped_column(default=0)
13     def __repr__(self):
14         return f"Producto({self.nombre!r}, {self.precio}, stock={self.stock})"
15
16     motor = create_engine("sqlite://")
17     Base.metadata.create_all(motor)
18
19     with Session(motor) as sesion:
20         sesion.add_all([Producto(nombre="Cable", precio=3.2, stock=150),
21                         Producto(nombre="Pintura", precio=12.5,
22                                 Producto(nombre="Rodillo", precio=6.0, stock=25)])
23         sesion.commit()
24         baratos = sesion.scalars(select(Producto).where(Producto.precio <
25                                     ↳ 10).order_by(Producto.precio)).all()
26         print(baratos)
27         pintura = sesion.scalar(select(Producto).filter_by(nombre="Pintura"))
28         pintura.stock += 10
29         sesion.commit()
30         print(pintura)

```

Salida real

```

[Producto('Cable', 3.2, stock=150), Producto('Rodillo', 6.0, stock=25)]
Producto('Pintura', 12.5, stock=10)

```

Consejos

- Pasa `echo=True` a `create_engine` para ver el SQL que genera el ORM: es la mejor forma de aprender.
- Cambiar de SQLite a PostgreSQL es, casi siempre, cambiar solo la URL del motor.

Ejercicio

Agrega un modelo Categoría y consulta los productos de una categoría ordenados por precio.

Práctica evaluable

Una sesión con commit y rollback · La lógica de la lección, sin librerías externas

Crea la clase `Sesion`, una versión mínima de la sesión de un ORM:

- `agregar(obj)`: lo deja pendiente.
- `pendientes`: propiedad con la cantidad de objetos pendientes.
- `commit()`: guarda los pendientes en la lista guardados, asignando a cada objeto (diccionario) un "id" correlativo desde 1, y vacía los pendientes. Devuelve cuántos guardó.
- `rollback()`: descarta los pendientes sin guardarlos.

Plantilla para empezar

```
1 class Sesion:
2     """Unidad de trabajo mínima, como Session de SQLAlchemy."""
3
4     def __init__(self):
5         self.guardados = []
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué hace `sesion.commit()`?

- Crea las tablas
- Borra la sesión
- Confirma en la base los cambios pendientes
- Cierra Python

8 Relaciones y consultas agregadas

`relationship()` conecta modelos: `cliente.pedidos` devuelve una lista de objetos y `pedido.cliente` el objeto relacionado. `back_populates` mantiene ambos lados sincronizados.

`cascade="all, delete-orphan"` hace que al borrar un cliente se borren sus pedidos.

Las consultas agregadas combinan `join`, `group_by` y funciones como `func.count` y `func.sum`.

Ejemplo

```

1  from sqlalchemy import create_engine, select, ForeignKey, func
2  from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship,
   ↪ Session
3
4  class Base(DeclarativeBase):
5      pass
6
7  class Cliente(Base):
8      __tablename__ = "clientes"
9      id: Mapped[int] = mapped_column(primary_key=True)
10     nombre: Mapped[str]
11     pedidos: Mapped[list["Pedido"]] = relationship(back_populates="cliente",
   ↪ cascade="all, delete-orphan")
12
13 class Pedido(Base):
14     __tablename__ = "pedidos"
15     id: Mapped[int] = mapped_column(primary_key=True)
16     total: Mapped[float]
17     cliente_id: Mapped[int] = mapped_column(ForeignKey("clientes.id"))
18     cliente: Mapped[Cliente] = relationship(back_populates="pedidos")
19
20 motor = create_engine("sqlite://")
21 Base.metadata.create_all(motor)
22 with Session(motor) as s:
23     ana = Cliente(nombre="Ana", pedidos=[Pedido(total=120), Pedido(total=80)])
24     luis = Cliente(nombre="Luis", pedidos=[Pedido(total=45)])
25     s.add_all([ana, luis])
26     s.commit()
27     print([p.total for p in ana.pedidos], "|", ana.pedidos[0].cliente.nombre)
28     consulta = (select(Cliente.nombre, func.count(Pedido.id),
   ↪ func.sum(Pedido.total))
29                 .join(Cliente.pedidos).group_by(Cliente.id)
30                 .order_by(func.sum(Pedido.total).desc()))
31     for fila in s.execute(consulta):
32         print(tuple(fila))

```

Salida real

```

[120.0, 80.0] | Ana
('Ana', 2, 200.0)
('Luis', 1, 45.0)

```

Consejos

- Cuidado con el problema N+1: usa `selectinload` para traer relaciones en una sola consulta.
- Las migraciones del esquema se manejan con Alembic, compañero natural de SQLAlchemy.

Ejercicio

Agrega productos y una tabla puente pedido_items, y calcula el producto más vendido con el ORM.

Práctica evaluable

Relaciones uno a muchos · La lógica de la lección, sin librerías externas

Con clientes (lista de (id, nombre)) y pedidos (lista de (cliente_id, total)):

- pedidos_por_cliente(clientes, pedidos): {nombre: [totales]}, incluidos los clientes sin pedidos (lista vacía), como cliente.pedidos en un ORM.
- ranking(clientes, pedidos): lista de (nombre, cantidad, suma) solo de clientes con pedidos, ordenada por suma de mayor a menor.

Plantilla para empezar

```
1 def pedidos_por_cliente(clientes, pedidos):
2     """{nombre: [totales]}"""
3
4
5 def ranking(clientes, pedidos):
6     """[(nombre, cantidad, suma)] de mayor a menor."""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Para qué sirve back_populates?

- Para cachear consultas
- Para crear índices
- Para mantener sincronizados ambos lados de una relación
- Para borrar datos

Misión del módulo

Reescribe tu agenda de clientes con SQLAlchemy, con relaciones y consultas agregadas.

Módulo 5

PostgreSQL y arquitectura de datos

Requisitos: N3.4 Type hints y dataclasses, N4.7 Patrones de diseño pythónicos. Aparece en el Nivel 4 del árbol.

9 PostgreSQL desde Python

PostgreSQL es la base de datos relacional de código abierto más completa: tipos avanzados (JSONB, fechas con zona horaria), concurrencia real y gran escalabilidad.

psycopg (versión 3) es el driver oficial. Usa %s como marcador de parámetros y conexiones como context manager.

RETURNING devuelve datos de la fila recién insertada, como su id, sin otra consulta.

Ejemplo

```

1  import os
2  import psycopg
3
4  DSN = os.environ["DATABASE_URL"] # p. ej.
   ↳ postgresql://usuario:clave@localhost:5432/tienda
5
6  with psycopg.connect(DSN) as con:
7      with con.cursor() as cur:
8          cur.execute("""
9              CREATE TABLE IF NOT EXISTS productos (
10                 id SERIAL PRIMARY KEY, nombre TEXT NOT NULL,
11                 precio NUMERIC(10, 2) NOT NULL, creado TIMESTAMPTZ DEFAULT
12                 ↳ now())""")
13          cur.execute("INSERT INTO productos (nombre, precio) VALUES (%s, %s)
14                 ↳ RETURNING id",
15                     ("Cable", 3.20))
16          nuevo_id = cur.fetchone()[0]
17          cur.execute("SELECT nombre, precio FROM productos WHERE id = %s",
18                     ↳ (nuevo_id,))
19          print(nuevo_id, cur.fetchone())
20  # al salir del with: commit automático, o rollback si hubo error

```

Resultado esperado

```
1 ('Cable', Decimal('3.20'))
```

Consejos

- Para dinero usa NUMERIC, nunca FLOAT: psycopg lo convierte en Decimal.
- Levanta PostgreSQL local en un minuto con Docker: `docker run -e POSTGRES_PASSWORD=clave -p 5432:5432 postgres`.

Ejercicio

Instala PostgreSQL (o usa Docker), crea tu modelo de tienda y migra los datos desde SQLite.

Práctica evaluable

Cadena de conexión segura · La lógica de la lección, sin librerías externas

Escribe `dsn(usuario, clave, base, host="localhost", puerto=5432)`, que devuelve `"postgresql://usuario:clave@host:puerto/base"`. La clave puede tener símbolos como `@`, `:` o `/` que romperían la URL: codifícala con `urllib.parse.quote(clave, safe="")`. Si el puerto no está entre 1 y 65535, lanza `ValueError`.

Plantilla para empezar

```
1 from urllib.parse import quote
2
3
4 def dsn(usuario, clave, base, host="localhost", puerto=5432):
5     """postgresql://usuario:clave@host:puerto/base"""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué tipo conviene para montos de dinero en PostgreSQL?

- a) TEXT
- b) FLOAT
- c) INTEGER siempre
- d) NUMERIC

10 Patrón Repository

El patrón Repository esconde la base de datos detrás de una interfaz: tu lógica de negocio pide 'guardar producto' sin saber si hay SQLite, PostgreSQL o una lista en memoria.

Con `typing.Protocol` defines el contrato y cualquier clase que lo cumpla sirve. Así pruebas la lógica con un repositorio en memoria, rápido y sin base real.

Junto con migraciones (Alembic) y pools de conexiones, es la base de las aplicaciones de datos profesionales.

Ejemplo

```

1  import sqlite3
2  from dataclasses import dataclass
3  from typing import Protocol
4
5  @dataclass
6  class Producto:
7      nombre: str
8      precio: float
9      id: int | None = None
10
11  class RepositorioProductos(Protocol):
12      def guardar(self, p: Producto) -> Producto: ...
13      def buscar_baratos(self, maximo: float) -> list[Producto]: ...
14
15  class RepoSQLite:
16      def __init__(self, con: sqlite3.Connection):
17          self.con = con
18          con.execute("CREATE TABLE IF NOT EXISTS productos (id INTEGER PRIMARY KEY,
19                      ↪ nombre TEXT, precio REAL)")
20      def guardar(self, p):
21          cur = self.con.execute("INSERT INTO productos (nombre, precio) VALUES (?,
22                      ↪ ?)", (p.nombre, p.precio))
23          self.con.commit()
24          p.id = cur.lastrowid
25          return p
26      def buscar_baratos(self, maximo):
27          filas = self.con.execute("SELECT nombre, precio, id FROM productos WHERE
28                      ↪ precio <= ? ORDER BY precio", (maximo,))
29          return [Producto(*f) for f in filas]
30
31  class RepoMemoria: # ideal para tests: sin base de datos
32      def __init__(self):
33          self.datos = []
34      def guardar(self, p):
35          p.id = len(self.datos) + 1
36          self.datos.append(p)
37          return p
38      def buscar_baratos(self, maximo):
39          return sorted((p for p in self.datos if p.precio <= maximo), key=lambda p:
40                      ↪ p.precio)
41
42  def cargar_catalogo(repo: RepositorioProductos) -> list[Producto]:
43      for nombre, precio in [("Cable", 3.2), ("Pintura", 12.5), ("Rodillo", 6.0)]:
44          repo.guardar(Producto(nombre, precio))
45      return repo.buscar_baratos(10)
46
47  print(cargar_catalogo(RepoSQLite(sqlite3.connect(":memory:"))))
48  print(cargar_catalogo(RepoMemoria()))

```

Salida real

```

[Producto(nombre='Cable', precio=3.2, id=1), Producto(nombre='Rodillo', precio=6.0, id=3)]
[Producto(nombre='Cable', precio=3.2, id=1), Producto(nombre='Rodillo', precio=6.0, id=3)]

```

Consejos

- La misma función produce el mismo resultado con dos repositorios distintos: esa es la prueba de que el diseño está desacoplado.
- Usa Alembic para versionar los cambios del esquema como si fueran commits.

Ejercicio

Crea RepoSQLAlchemy que cumpla el mismo Protocol y verifica con pytest que los tres repositorios se comportan igual.

Práctica evaluable

Repositorio en memoria · La lógica de la lección, sin librerías externas

Ya existe la dataclass Producto(nombre, precio, id=None). Crea la clase RepoMemoria, que cumple el mismo contrato que un repositorio de base de datos:

- guardar(p): le asigna un id correlativo desde 1 (si no tenía), lo guarda y lo devuelve.
- obtener(id): el producto con ese id, o None.
- buscar_baratos(maximo): los productos con precio menor o igual al máximo, ordenados por precio.
- len(repo) devuelve cuántos productos hay.

Datos que prepara el evaluador (también prueba con otros valores)

```

1  from dataclasses import dataclass
2
3
4  @dataclass
5  class Producto:
6      nombre: str
7      precio: float
8      id: object = None

```

Plantilla para empezar

```

1  class RepoMemoria:
2      """Repositorio de productos sin base de datos (ideal para tests)."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué ventaja da el patrón Repository?

- Cambiar o simular la base sin tocar la lógica de negocio
- Consultas más cortas
- No necesitar SQL nunca
- Más velocidad siempre

Misión del módulo

Migra tu tienda a PostgreSQL con un repositorio intercambiable y tests sin base real.

Módulo 6

Análisis con DuckDB

Requisitos: N4.5 Profiling y optimización, N2.4 Archivos, CSV y JSON. Aparece en el Nivel 4 del árbol.

11 Consultar archivos directamente

DuckDB es una base analítica que corre dentro de tu programa, sin servidor. Está optimizada para agregaciones sobre millones de filas.

Puede consultar archivos CSV, Parquet o JSON directamente con SQL, sin importarlos primero: `SELECT ... FROM 'ventas.csv'`.

Detecta tipos automáticamente (fechas, números) y aprovecha todos los núcleos del procesador.

Ejemplo

```

1 import duckdb, csv, random
2
3 random.seed(3)
4 with open("ventas.csv", "w", newline="", encoding="utf-8") as f:
5     w = csv.writer(f)
6     w.writerow(["fecha", "sucursal", "producto", "monto"])
7     for _ in range(10_000):
8         w.writerow([f"2026-{random.randint(1, 12):02d}-15",
9                     random.choice(["Centro", "Norte", "Sur"]),
10                    random.choice(["Cable", "Pintura", "Rodillo"]),
11                    round(random.uniform(5, 200), 2)])
12
13 resultado = duckdb.sql("""
14     SELECT sucursal, COUNT(*) AS ventas, ROUND(SUM(monto), 2) AS total,
15     ↪ ROUND(AVG(monto), 2) AS ticket
16     FROM 'ventas.csv'
17     GROUP BY sucursal ORDER BY total DESC
18 """)
19 print(resultado.fetchall())
20 print(duckdb.sql("SELECT MONTH(fecha) AS mes, ROUND(SUM(monto)) FROM 'ventas.csv'
21 ↪ GROUP BY mes ORDER BY mes LIMIT 3").fetchall())

```

Salida real

```

[('Centro', 3358, 349917.38, 104.2), ('Norte', 3358, 341397.07, 101.67), ('Sur', 3284, 333626.94, 101.59)
[(1, 87565.0), (2, 83075.0), (3, 84116.0)]

```

Consejos

- Usa comodines para consultar muchos archivos a la vez: `FROM 'ventas_*.csv'`.
- `duckdb.sql(...).show()` imprime una tabla bonita en la consola.

Ejercicio

Genera 1 millón de filas y calcula ventas por mes y sucursal; mide el tiempo con `time.perf_counter`.

Práctica evaluable**Agregaciones como en DuckDB** · La lógica de la lección, sin librerías externas

Escribe `agrupar(filas, clave, valor)`: con una lista de diccionarios, devuelve `{grupo: {"cantidad": n, "total": suma, "promedio": promedio}}`, con total y promedio redondeados a 2 decimales. Es lo que calcula `SELECT clave, COUNT(*), SUM(valor), AVG(valor) ... GROUP BY clave`.

Plantilla para empezar

```
1 def agrupar(filas, clave, valor):
2     """GROUP BY clave con COUNT, SUM y AVG."""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué distingue a DuckDB de PostgreSQL?

- a) Son iguales
- b) DuckDB corre dentro de tu programa y está optimizada para análisis
- c) DuckDB no usa SQL
- d) DuckDB solo lee Excel

12 DuckDB, pandas y Parquet

Parquet es un formato columnar comprimido: ocupa mucho menos que un CSV y se lee mucho más rápido. Es el estándar en ingeniería de datos.

DuckDB consulta un DataFrame de pandas por su nombre de variable y devuelve resultados como DataFrame con `.df()`.

La combinación ideal: pandas para manipular, DuckDB para agregar con SQL, Parquet para guardar.

Ejemplo

```
1 import duckdb
2 import pandas as pd
3
4 df = pd.DataFrame({"producto": ["Cable", "Pintura", "Cable", "Rodillo", "Pintura"],
5                     "cantidad": [10, 2, 5, 3, 4],
6                     "precio": [3.2, 12.5, 3.2, 6.0, 12.5]})
7
8 duckdb.sql("COPY (SELECT * FROM df) TO 'ventas.parquet' (FORMAT parquet)")
9 resumen = duckdb.sql("""
10     SELECT producto, SUM(cantidad) AS unidades, SUM(cantidad * precio) AS ingreso
11     FROM 'ventas.parquet' GROUP BY producto ORDER BY ingreso DESC
12 """).df()
13 print(resumen)
14 print(type(resumen).__name__, "| columnas:", list(resumen.columns))
```

Salida real

```

    producto  unidades  ingreso
0  Pintura      6.0      75.0
1   Cable     15.0      48.0
2  Rodillo      3.0      18.0
DataFrame | columnas: ['producto', 'unidades', 'ingreso']

```

Consejos

- Polars es otra opción muy rápida que también lee y escribe Parquet.
- Guarda en Parquet particionado por fecha cuando los datos crecen.

Ejercicio

Convierte tu CSV de 1 millón de filas a Parquet, compara tamaños en disco y tiempos de consulta.

Práctica evaluable

Filas y columnas (formato columnar) · La lógica de la lección, sin librerías externas

Parquet y DuckDB guardan los datos por columnas en lugar de por filas. Escribe:

- `a_columnas(filas)`: de una lista de diccionarios con las mismas claves a `{columna: [valores]}`.
- `a_filas(columnas)`: el camino inverso.
- `suma_columna(columnas, nombre)`: la suma de una columna, sin recorrer las demás.

Plantilla para empezar

```

1  def a_columnas(filas):
2      """[{"a": 1, "b": 2}, {"a": 3, "b": 4}] -> {"a": [1, 3], "b": [2, 4]}"""
3
4
5  def a_filas(columnas):
6      """El camino inverso."""
7
8
9  def suma_columna(columnas, nombre):
10     """Suma de una sola columna."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Por qué Parquet es más eficiente que CSV para análisis?

- Lo abre Excel
- No tiene tipos
- Es texto plano
- Es columnar y comprimido

Misión del módulo

Analiza un año de ventas (millones de filas) con DuckDB y compara tiempos contra pandas.

Proyecto integrador

Sistema de inventario con reportes

Un sistema de inventario multisucursal con modelo relacional, capa de repositorio, ORM, reportes analíticos y pruebas automatizadas.

Paso a paso

1. Diseña el modelo (sucursales, productos, movimientos, pedidos) y dibuja su diagrama.
2. Implementa los modelos con SQLAlchemy y migraciones con Alembic.
3. Crea una capa Repository y la lógica de negocio (entradas, salidas, transferencias).
4. Agrega índices según los planes de consulta de los reportes.
5. Genera reportes analíticos con DuckDB a partir de una exportación Parquet.
6. Cubre la lógica con tests usando el repositorio en memoria.

Criterios de éxito

Funciona sin errores con entradas normales y extrañas, está dividido en funciones o clases con nombres claros, tiene docstrings y tests donde corresponde, y registraste el proceso en la Plantilla de proyecto del Cuadernillo.

Checklist de cierre

- ☐ Misión cumplida: SQL esencial
- ☐ Misión cumplida: sqlite3 desde Python
- ☐ Misión cumplida: Modelado de datos
- ☐ Misión cumplida: SQLAlchemy ORM
- ☐ Misión cumplida: PostgreSQL y arquitectura de datos
- ☐ Misión cumplida: Análisis con DuckDB
- ☐ Terminé el proyecto "Sistema de inventario con reportes"

Solucionario de autoevaluación

- 1. SELECT, WHERE y ORDER BY:** b) WHERE. WHERE conserva solo las filas que cumplen la condición.
- 2. JOIN y GROUP BY:** d) LEFT JOIN desde sucursales. LEFT JOIN conserva todas las filas de la tabla de la izquierda.
- 3. Parámetros seguros y filas como diccionarios:** c) Con parámetros ? y valores separados. El motor trata los parámetros como datos, nunca como código SQL.
- 4. Transacciones y errores:** c) Se revierte también. La transacción es atómica: todo o nada.
- 5. Tablas, claves y relaciones:** d) Con una tabla puente con dos claves foráneas. pedido_items une pedidos y productos y guarda datos propios, como la cantidad.
- 6. Índices y planes de consulta:** b) Que recorre toda la tabla. SCAN es el escaneo completo; SEARCH ... USING INDEX es la búsqueda indexada.
- 7. Modelos y sesiones:** c) Confirma en la base los cambios pendientes. Hasta el commit, los cambios viven solo en la sesión.
- 8. Relaciones y consultas agregadas:** c) Para mantener sincronizados ambos lados de una relación. Al agregar un pedido a cliente.pedidos, pedido.cliente se actualiza solo.
- 9. PostgreSQL desde Python:** d) NUMERIC. NUMERIC es exacto; FLOAT acumula errores de redondeo.
- 10. Patrón Repository:** a) Cambiar o simular la base sin tocar la lógica de negocio. La lógica depende de un contrato, no de una base concreta.
- 11. Consultar archivos directamente:** b) DuckDB corre dentro de tu programa y está optimizada para análisis. PostgreSQL es un servidor para muchas transacciones; DuckDB, un motor embebido para analítica.
- 12. DuckDB, pandas y Parquet:** d) Es columnar y comprimido. Leer solo las columnas necesarias, ya comprimidas y tipadas, es mucho más rápido.

Pistas y soluciones de las prácticas

Usa las pistas de a una, y solo después de intentarlo. Hay más de una solución correcta: si la tuya pasa las pruebas, compárala con esta para aprender otra forma de pensarlo.

1. SELECT, WHERE y ORDER BY · Un SELECT escrito a mano

Pista 1: El orden de las operaciones es el del motor: primero WHERE, después ORDER BY y al final SELECT de columnas.

Pista 2: `dict(f)` copia una fila.

```
1 def seleccionar(tabla, columnas=None, donde=None, orden=None, desc=False):
2     """SELECT columnas FROM tabla WHERE donde ORDER BY orden"""
3     filas = [f for f in tabla if donde is None or donde(f)]
4     if orden:
5         filas = sorted(filas, key=lambda f: f[orden], reverse=desc)
6     if columnas:
7         return [{c: f[c] for c in columnas} for f in filas]
8     return [dict(f) for f in filas]
```

Por qué funciona: Así trabaja un motor de base de datos, en ese orden: filtra, ordena y proyecta columnas. Escribirlo una vez a mano hace que SQL deje de ser magia.

2. JOIN y GROUP BY · LEFT JOIN y GROUP BY a mano

Pista 1: Crea primero un grupo vacío por sucursal.

Pista 2: Ordenar por `-total` da orden descendente sin perder el desempate ascendente por nombre.

```
1 def totales_por_sucursal(sucursales, ventas):
2     """Equivale a un LEFT JOIN con GROUP BY."""
3     grupos = {sid: [nombre, 0, 0] for sid, nombre in sucursales}
4     for sid, monto in ventas:
5         if sid in grupos:
6             grupos[sid][1] += 1
7             grupos[sid][2] += monto
8     filas = [tuple(g) for g in grupos.values()]
9     return sorted(filas, key=lambda t: (-t[2], t[0]))
```

Por qué funciona: Arrancar con todas las sucursales en cero es exactamente lo que hace el LEFT JOIN; recorrer las ventas acumulando es el GROUP BY con COUNT y SUM.

3. Parámetros seguros y filas como diccionarios · Consultas parametrizadas seguras

Pista 1: Un `?` por cada filtro, unidos con `" AND "`.

Pista 2: `tuple(filtros.values())` conserva el mismo orden que las columnas.

```

1 def armar_consulta(tabla, filtros, permitidas):
2     """-> (sql, parametros)"""
3     if tabla not in permitidas:
4         raise ValueError(f"Tabla no permitida: {tabla}")
5     for columna in filtros:
6         if not columna.isidentifier():
7             raise ValueError(f"Columna inválida: {columna}")
8     sql = f"SELECT * FROM {tabla}"
9     if filtros:
10        sql += " WHERE " + " AND ".join(f"{c} = ?" for c in filtros)
11    return sql, tuple(filtros.values())

```

Por qué funciona: El texto SQL solo contiene nombres validados contra una lista o una regla; todo dato del usuario viaja como parámetro. Esa separación es la defensa completa contra la inyección SQL.

4. Transacciones y errores · Una transferencia atómica

Pista 1: Primero todas las validaciones, después los cambios.

Pista 2: Si validas a mitad de camino, podrías dejar una cuenta modificada y la otra no.

```

1 def transferir(cuentas, origen, destino, monto):
2     """Todo o nada, como una transacción."""
3     if monto <= 0:
4         raise ValueError("El monto debe ser positivo")
5     for titular in (origen, destino):
6         if titular not in cuentas:
7             raise ValueError(f"No existe la cuenta de {titular}")
8     if cuentas[origen] < monto:
9         raise ValueError(f"Saldo insuficiente en la cuenta de {origen}")
10    cuentas[origen] -= monto
11    cuentas[destino] += monto
12    return cuentas

```

Por qué funciona: Validar todo antes de escribir es la versión manual de una transacción. La base de datos lo garantiza aunque se corte la luz a mitad: por eso existen commit y rollback.

5. Tablas, claves y relaciones · Integridad referencial

Pista 1: Un set de ids hace instantánea la comprobación.

Pista 2: La tabla puente se recorre filtrando por pedido.

```

1 def claves_rotas(productos, categorias):
2     """Productos con categoría inexistente."""
3     ids = {c["id"] for c in categorias}
4     return [p["nombre"] for p in productos if p["categoria_id"] not in ids]
5
6
7 def detalle_pedido(pedido_id, items, productos):
8     """[(nombre, cantidad)] de un pedido."""
9     nombres = {p["id"]: p["nombre"] for p in productos}
10    return [(nombres[prod], cant) for ped, prod, cant in items if ped == pedido_id]

```

Por qué funciona: Una FOREIGN KEY hace esta verificación en cada inserción. La tabla puente resuelve el muchos a muchos: un pedido tiene muchos productos y un producto aparece en muchos pedidos.

6. Índices y planes de consulta · Un índice a mano

Pista 1: enumerate da la posición de cada fila.

Pista 2: Acceder por posición (filas[i]) no recorre la lista.

```

1  def crear_indice(filas, columna):
2      """{valor: [posiciones]}"""
3      indice = {}
4      for posicion, fila in enumerate(filas):
5          indice.setdefault(fila[columna], []).append(posicion)
6      return indice
7
8
9  def buscar(filas, indice, valor):
10     """Filas con ese valor, usando el índice."""
11     return [filas[i] for i in indice.get(valor, [])]
```

Por qué funciona: Construir el índice cuesta un recorrido completo; después, cada búsqueda va directo a las filas. Es el mismo intercambio que hace CREATE INDEX: escrituras un poco más lentas a cambio de lecturas mucho más rápidas.

7. Modelos y sesiones · Una sesión con commit y rollback

Pista 1: Guarda los pendientes en un atributo privado, self._pendientes.

Pista 2: Lleva el próximo id en un contador de la sesión.

```

1  class Sesion:
2      """Unidad de trabajo mínima, como Session de SQLAlchemy."""
3
4      def __init__(self):
5          self.guardados = []
6          self._pendientes = []
7          self._proximo_id = 1
8
9      def agregar(self, obj):
10         self._pendientes.append(obj)
11
12     @property
13     def pendientes(self):
14         return len(self._pendientes)
15
16     def commit(self):
17         for obj in self._pendientes:
18             obj["id"] = self._proximo_id
19             self._proximo_id += 1
20             self.guardados.append(obj)
21         cantidad = len(self._pendientes)
22         self._pendientes = []
23         return cantidad
24
25     def rollback(self):
26         self._pendientes = []
```

Por qué funciona: Los cambios viven en la sesión hasta el commit; un rollback los descarta sin tocar lo guardado. Así funciona la unidad de trabajo de SQLAlchemy.

8. Relaciones y consultas agregadas · Relaciones uno a muchos

Pista 1: `dict(clientes)` convierte la lista de pares en `{id: nombre}`.

Pista 2: Reutiliza `pedidos_por_cliente` en `ranking`.

```

1  def pedidos_por_cliente(clientes, pedidos):
2      """{nombre: [totales]}"""
3      nombres = dict(clientes)
4      resultado = {nombre: [] for _, nombre in clientes}
5      for cid, total in pedidos:
6          resultado[nombres[cid]].append(total)
7      return resultado
8
9
10 def ranking(clientes, pedidos):
11     """[(nombre, cantidad, suma)] de mayor a menor."""
12     filas = [(n, len(t), sum(t)) for n, t in pedidos_por_cliente(clientes,
13     ↪ pedidos).items() if t]
14     return sorted(filas, key=lambda f: f[2], reverse=True)

```

Por qué funciona: La primera función es la relación (relationship) del ORM: desde el cliente llegas a sus pedidos. La segunda es la consulta agregada con JOIN, GROUP BY y ORDER BY.

9. PostgreSQL desde Python · Cadena de conexión segura

Pista 1: Sin codificar, la `@` de la clave parecería el separador del host.

Pista 2: `safe=""` hace que `quote` codifique también `/`.

```

1  from urllib.parse import quote
2
3
4  def dsn(usuario, clave, base, host="localhost", puerto=5432):
5      """postgresql://usuario:clave@host:puerto/base"""
6      if not 1 <= puerto <= 65535:
7          raise ValueError(f"Puerto inválido: {puerto}")
8      return f"postgresql://{quote(usuario, safe='')}:{quote(clave,
8      ↪ safe='')}@{host}:{puerto}/{base}"

```

Por qué funciona: Una clave con símbolos es el error más común al conectarse a PostgreSQL. En tu proyecto real, esta cadena se arma con datos de variables de entorno, nunca escritos en el código.

10. Patrón Repository · Repositorio en memoria

Pista 1: Un diccionario `{id: producto}` hace instantáneo obtener.

Pista 2: `.get(id_)` devuelve `None` si no existe.

```

1 class RepoMemoria:
2     """Repositorio de productos sin base de datos (ideal para tests)."""
3
4     def __init__(self):
5         self._datos = {}
6
7     def guardar(self, p):
8         if p.id is None:
9             p.id = len(self._datos) + 1
10            self._datos[p.id] = p
11            return p
12
13    def obtener(self, id_):
14        return self._datos.get(id_)
15
16    def buscar_baratos(self, maximo):
17        return sorted((p for p in self._datos.values() if p.precio <= maximo),
18            ↪ key=lambda p: p.precio)
19
20    def __len__(self):
21        return len(self._datos)

```

Por qué funciona: Este repositorio cumple el mismo contrato que uno con SQLite o PostgreSQL. La lógica de negocio que lo usa no nota la diferencia, y sus tests corren en milisegundos.

11. Consultar archivos directamente · Agregaciones como en DuckDB

Pista 1: Guarda por grupo un par [cantidad, suma].

Pista 2: El promedio es suma / cantidad, calculado después de recorrer.

```

1 def agrupar(filas, clave, valor):
2     """GROUP BY clave con COUNT, SUM y AVG."""
3     acumulado = {}
4     for fila in filas:
5         g = acumulado.setdefault(fila[clave], [0, 0])
6         g[0] += 1
7         g[1] += fila[valor]
8     return {k: {"cantidad": n, "total": round(s, 2), "promedio": round(s / n, 2)}
9         ↪ for k, (n, s) in acumulado.items()}

```

Por qué funciona: Un solo recorrido acumula lo necesario y el promedio se deriva al final: así lo hacen los motores analíticos, que nunca guardan todas las filas de cada grupo.

12. DuckDB, pandas y Parquet · Filas y columnas (formato columnar)

Pista 1: Las claves de la primera fila definen las columnas.

Pista 2: `zip(*columnas.values())` recorre las columnas en paralelo.

```
1 def a_columnas(filas):
2     """Lista de filas -> diccionario de columnas."""
3     if not filas:
4         return {}
5     return {c: [f[c] for f in filas] for c in filas[0]}
6
7
8 def a_filas(columnas):
9     """Diccionario de columnas -> lista de filas."""
10    if not columnas:
11        return []
12    nombres = list(columnas)
13    return [dict(zip(nombres, valores)) for valores in zip(*columnas.values())]
14
15
16 def suma_columna(columnas, nombre):
17     """Suma de una sola columna."""
18     return sum(columnas[nombre])
```

Por qué funciona: En formato columnar, sumar una columna lee solo esa lista; en filas, habría que tocar cada fila completa. Con millones de filas y decenas de columnas, esa es la diferencia entre segundos y minutos.